PROJECT_DOCUMENTATION.md

Sistema de Gestión de RRHH - Fichaje CTI

Tabla de Contenidos

1. Descripción de Módulos
2. Mapa de Relaciones de Entidades
3. Lógica de Negocio: Horarios Rotativos y Fichaje
4. Pasos para Integración Frontend-Backend

Descripción de Módulos

Auth (Autenticación)

Propósito: Gestión de autenticación y autorización del sistema

- **Login**: Email/DNI + Password → JWT con payload `{id, rol, empresaId}`
- **Roles**: SUPERADMIN, ADMIN, WORKER
- **Guards**: `JwtAuthGuard` + `RolesGuard` para protección de endpoints
- **Decoradores**: `@GetUser()` para extracción de usuario autenticado
- **Tokens**: HTTP-only cookies + response body para desarrollo

Users (Usuarios)

Propósito: Gestión completa del ciclo de vida de usuarios

- **Creación**: Transaccional con validación de roles y empresa
- **Roles**: SuperAdmin crea Admins, Admins crean Workers
- **Validaciones**: Email único, DNI formato oficial, teléfono único
- **Relaciones**: Empresa, Horario, Worker, Activity
- **Seguridad**: Filtrado estricto por `empresaId`

Company (Empresas)

Propósito: Gestión multi-tenancy del sistema

- **Creación**: Transaccional empresa + admin asociado
- **Datos**: Nombre, CIF, dirección, coordenadas GPS, radio permitido
- **Configuración**: Días de archivo para incidencias/solicitudes
- **Relaciones**: Central en el sistema (1:N con todos los módulos)

Workday (Fichaje)

Propósito: Sistema de control de jornada laboral

- **Check-in/Out**: Geolocalización con validación de radio

- **Estados**: Activa, finalizada, historial
- **Geofencing**: Alarmas automáticas por zona no autorizada
- **Cálculos**: Minutos trabajados, detección de retrasos
- **Validaciones**: Anti-duplicación de jornadas activas

📅 Schedule (Horarios)

Propósito: Gestión de horarios laborales complejos

- **Tipos**: FIJO, ROTATIVO, PARTIDO
- **Rotación**: Configurable por número de semanas
- **Estructura**: Horario → Distribución Día → Franja Horaria
- **Validaciones**: Anti-solapamiento de franjas
- **Soft Delete**: Mantenimiento de auditoría

🕐 TimeSlot (Franjas Horarias)

Propósito: Gestión de bloques de tiempo dentro de horarios

- **Estructura**: Hora inicio/fin con validación de rangos
- **Relaciones**: Pertenece a Distribución Día
- **Validaciones**: Solapamiento, orden cronológico

📊 Attendance (Asistencia)

Propósito: Registro diario de presencia/ausencia

- **Registro**: Presente/Ausente con motivo opcional
- **Unicidad**: Un registro por usuario por día
- **Historial**: Consultas por usuario y por empresa
- **Validaciones**: Anti-duplicación diaria

🏖️ Absence (Ausencias)

Propósito: Gestión de solicitudes de ausencia

- **Tipos**: VACACIONES, ENFERMEDAD, PERSONAL
- **Estados**: PENDIENTE → APROBADA/RECHAZADA
- **Balance**: Cálculo automático de días disponibles (22 por ley)
- **Validaciones**: Anti-solapamiento de fechas

📋 Request (Solicitudes)

Propósito: Sistema de solicitudes laborales

- **Workflow**: Worker solicita → Admin revisa → Aprueba/Rechaza
- **Tipos**: Cambios horarios, permisos, etc.
- **Transaccional**: Creación + posibles acciones asociadas
- **Historial**: Registro completo de revisiones

⚠️ Issue (Incidencias)

Propósito: Registro de incidencias laborales

- **Tipos**: Retrasos, ausencias injustificadas, etc.
- **Automáticas**: Generadas por sistema de fichaje
- **Manuales**: Reportadas por usuarios/admins
- **Limpieza**: Automática por antigüedad configurable

🔔 Alarms (Alarmas)

Propósito: Sistema de alertas en tiempo real

- **Tipos**: UNAUTHORIZED_ZONE, LATE_CHECKIN, etc.
- **WebSocket**: Notificaciones instantáneas a admins
- **Anti-spam**: Prevención de duplicados
- **Resolución**: Marcado como leídas/resueltas

🎽 Activity (Puestos/Actividades)

Propósito: Catálogo de puestos de trabajo

- **Gestión**: Creación, actualización, desactivación
- **Estructura**: Tipo, departamento, descripción
- **Relaciones**: Usuarios asignados
- **Estados**: Activo/Inactivo

🏭 Worker (Trabajadores)

Propósito: Información laboral detallada

- **Datos**: Contrato, puesto, departamento
- **Relaciones**: Usuario, Horario, Activity
- **Gestión**: Actualización de datos laborales

📱 Allowed-Phones (Teléfonos Autorizados)

Propósito: Control de dispositivos permitidos

- **Validación**: Lista blanca de números
- **Uso**: Para acceso por SMS/código
- **Administración**: CRUD por empresa

🔑 Codigo-Acceso (Códigos de Acceso)

Propósito: Códigos temporales de acceso

- **Generación**: Códigos únicos por tiempo
- **Validación**: Verificación y expiración
- **Uso**: Para recuperación de cuenta, acceso temporal

🌱 Seed (Siembra de Datos)

Propósito: Inicialización de datos del sistema

- **Datos base**: Empresas, usuarios, horarios ejemplo
- **Entorno**: Desarrollo y pruebas
- **Ejecución**: Comandos controlados

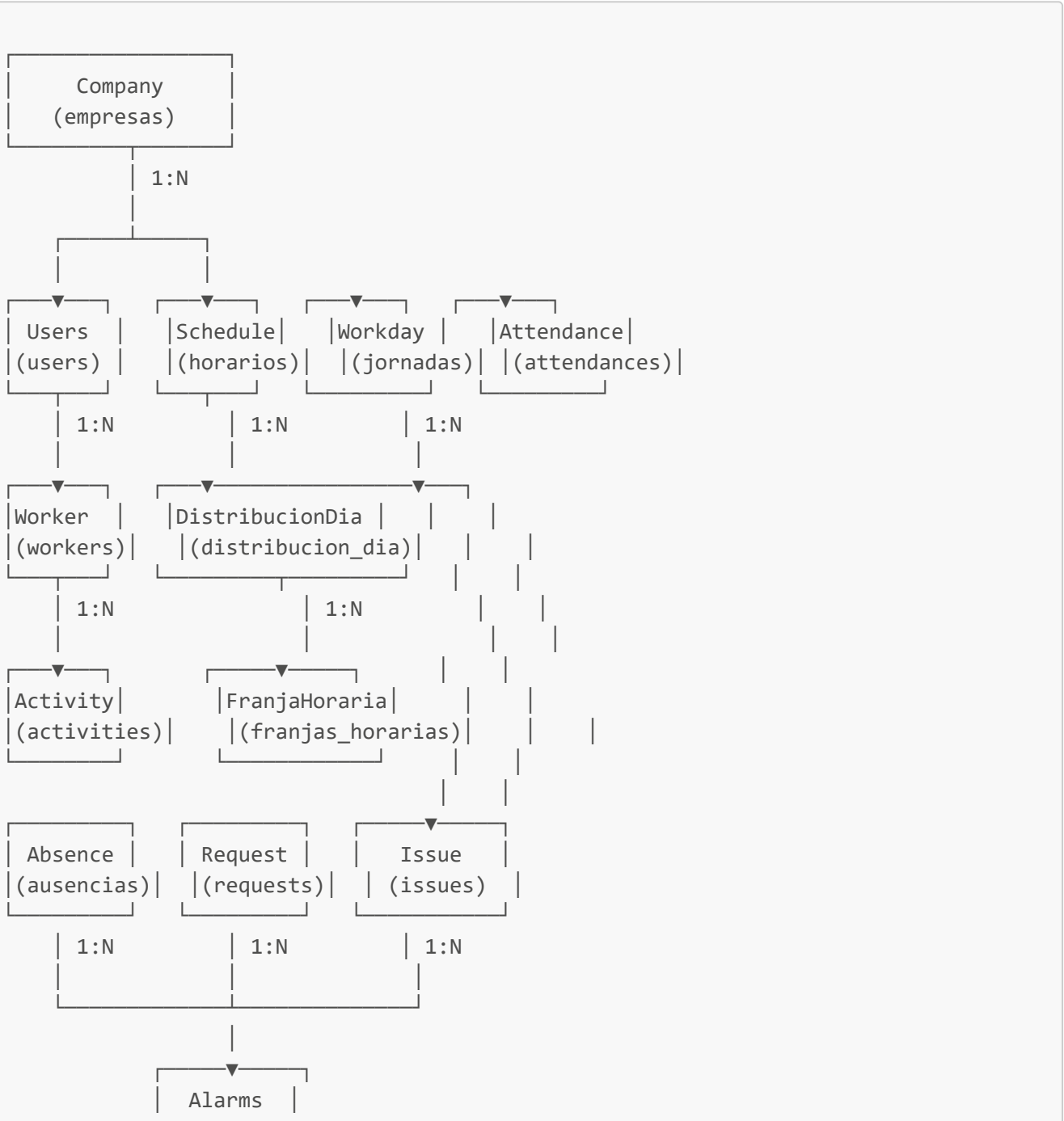
🔧 Common (Utilidades Comunes)

Propósito: Funcionalidades compartidas

- **BaseCleanup**: Servicio abstracto para limpieza
- **Interfaces**: Contratos comunes
- **Utilidades**: Helpers reutilizables

🗺️ Mapa de Relaciones de Entidades

📊 Diagrama de Relaciones Principales



```
                │ (alarms)  │
                └──────────┘
```

Relaciones Detalladas

Company (Empresa) - Entidad Central

- **Users** (1:N): `usuarios: User[]`
- **Schedule** (1:N): `horarios: Horario[]`
- **Request** (1:N): `solicitudes: Request[]`
- **Attendance** (1:N): `asistencias: Attendance[]`

Users (Usuarios)

- **Company** (N:1): `empresa: Company | null`
- **Schedule** (N:1): `horario: Horario | null`
- **Worker** (1:1): `worker: Worker`
- **Workday** (1:N): `jornadas: Workday[]`
- **Attendance** (1:N): `attendances: Attendance[]`
- **Issue** (1:N): `incidencias: Issue[]`
- **Absence** (1:N): `ausencias: Ausencia[]`
- **Activity** (N:1): `actividad: Activity`

Schedule (Horarios)

- **Company** (N:1): `empresa: Company`
- **DistribucionDia** (1:N): `distribuciones: DistribucionDia[]`
- **Worker** (1:N): `usuarios: Worker[]`

DistribucionDia (Distribución por Día)

- **Schedule** (N:1): `horario: Horario`
- **FranjaHoraria** (1:N): `franjas: FranjaHoraria[]`
- **Campos clave**: `diaSemana`, `horasDia`, `semanaRotacion`

FranjaHoraria (Franjas Horarias)

- **DistribucionDia** (N:1): `distribucionDia: DistribucionDia`
- **Campos**: `inicio: time`, `fin: time`

Workday (Jornadas)

- **User** (N:1): `usuario: User`
- **Company** (N:1): `empresa: Company`
- **Campos**: `inicio`, `fin`, `latitudEntrada`, `longitudEntrada`

Lógica de Negocio: Horarios Rotativos y Fichaje

🔄 Sistema de Horarios Rotativos

Estructura Jerárquica

```
Horario (Padre)
├── DistribucionDia (Hijos - 7 días)
│   ├── LUNES (semanaRotacion: 1)
│   ├── MARTES (semanaRotacion: 1)
│   └── ...
├── DistribucionDia (Hijos - 7 días)
│   ├── LUNES (semanaRotacion: 2)
│   ├── MARTES (semanaRotacion: 2)
│   └── ...
└── FranjaHoraria (Nietos)
    ├── inicio: "08:00"
    ├── fin: "14:00"
    └── ...
```

Configuración de Rotación

- **esRotativo**: Boolean que activa el modo rotativo
- **semanasRotacion**: Número de semanas en el ciclo (ej: 2)
- **semanaRotacion**: En cada DistribucionDia, indica a qué semana pertenece

Algoritmo de Rotación

1. **Semana 1**: Aplica distribuciones con `semanaRotacion = 1`
2. **Semana 2**: Aplica distribuciones con `semanaRotacion = 2`
3. **Ciclo**: Después de la semana N, vuelve a la semana 1
4. **Cálculo**: `(semanaActual - 1) % semanasRotacion + 1`

Validaciones Implementadas

- **Anti-solapamiento**: Franjas no pueden superponerse en el mismo día
- **Integridad**: Cascading deletes para mantener consistencia
- **Horas totales**: Validación de horas por día
- **Días laborables**: Control de qué días son trabajables

⏰ Sistema de Fichaje

Flujo de Check-in

1. **Validación de Usuario**:
   - Usuario autenticado con `empresaId`
   - Sin jornada activa (`fin = null`)
2. **Geolocalización**:

- Recibe `latitud`, `longitud` del cliente
- Compara con coordenadas de la empresa
- Calcula distancia usando fórmula Haversine
- Radio permitido: `empresa.radioPermitido` (default: 600m)

3. **Alarmas Geofencing**:
   - Si distancia > radio permitido → `UNAUTHORIZED_ZONE`
   - Registro automático en sistema de alarmas
4. **Creación de Jornada**:

```
{
  inicio: new Date(),
  latitudEntrada: dto.latitud,
  longitudEntrada: dto.longitud,
  tipo: 'Entrada',
  usuario: { id: user.id },
  empresa: { id: user.empresaId }
}
```

5. **Validación de Retraso**:
   - Compara hora actual con primera franja del horario
   - Si es tarde → `LATE_CHECKIN` alarm

Flujo de Check-out

1. **Búsqueda de Jornada Activa**:
   - Usuario con `fin = null`
   - Última jornada sin cerrar
2. **Geolocalización**:
   - Misma validación que check-in
   - Coordenadas de salida
3. **Cálculos Automáticos**:
   - `total_minutos = fin - inicio`
   - Duración en minutos trabajados
4. **Cierre de Jornada**:

```
{
  fin: new Date(),
  latitudSalida: dto.latitud,
  longitudSalida: dto.longitud,
  total_minutos: calculatedMinutes,
```

```
    tipo: 'Salida'
  }
```

Sistema de Alarmas Integrado

- **Tipos de Alarmas**:
  - `UNAUTHORIZED_ZONE`: Fichaje fuera de radio permitido
  - `LATE_CHECKIN`: Fichaje después de hora de inicio
  - `ABSENCE`: No fichó en el día
  - `MULTIPLE_CHECKIN`: Intentos de fichaje duplicado
- **Resolución Automática**:
  - Al hacer check-in, resuelve alarmas `ABSENCE`
  - Sistema anti-spam para evitar duplicados

Validaciones de Negocio

- **Única Jornada Activa**: Un usuario solo puede tener una jornada sin cerrar
- **Geofencing**: Validación estricta de ubicación
- **Horarios Laborables**: Solo permite fichar en días laborables
- **Integridad de Datos**: Transacciones para operaciones críticas

---

🔌 Pasos para Integración Frontend-Backend

📋 1. Configuración Inicial

Variables de Entorno

```
# Backend API
VITE_API_URL=http://localhost:3000/api

# Tiempo de expiración de tokens
VITE_TOKEN_EXPIRY_CHECK=60000

# Configuración de geolocalización
VITE_DEFAULT_GEOFENCE_RADIUS=600
```

Configuración de Axios

```
// src/api/axios.ts
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
```

```
  withCredentials: true, // Para cookies HTTP-only
  headers: {
    'Content-Type': 'application/json',
  },
});

// Interceptor para refresh de tokens
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    if (error.response?.status === 401) {
      // Redirigir a login
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

🔐 2. Sistema de Autenticación

Login y Gestión de Tokens

```
// src/services/auth.service.ts
export const login = async (credentials: LoginDto) => {
  try {
    const response = await api.post('/auth/login', credentials);

    // El backend establece la cookie automáticamente
    // Guardamos datos del usuario en estado global
    setUser(response.data.user);

    return response.data;
  } catch (error) {
    throw new Error(error.response?.data?.message || 'Error de login');
  }
};

export const logout = async () => {
  await api.post('/auth/logout');
  clearUser();
  window.location.href = '/login';
};
```

Estado Global de Usuario

```
// src/stores/auth.store.ts
interface UserState {
  user: UserPayload | null;
```

```
  empresaId: number | null;
  rol: UserRole | null;
}

export const useAuthStore = create<UserState>((set) => ({
  user: null,
  empresaId: null,
  rol: null,

  setUser: (userData) => set({
    user: userData,
    empresaId: userData.empresaId,
    rol: userData.rol,
  }),

  clearUser: () => set({
    user: null,
    empresaId: null,
    rol: null,
  }),
}));
```

3. Mapeo de DTOs y Validaciones

Ejemplo: Formulario de Fichaje

```
// src/types/workday.types.ts
export interface CheckInForm {
  latitud: number;
  longitud: number;
  presente: boolean;
}

// Validación frontend que coincide con backend DTO
export const validateCheckIn = (data: CheckInForm): string[] => {
  const errors: string[] = [];

  if (!data.latitud || data.latitud < -90 || data.latitud > 90) {
    errors.push('La latitud debe estar entre -90 y 90');
  }

  if (!data.longitud || data.longitud < -180 || data.longitud > 180) {
    errors.push('La longitud debe estar entre -180 y 180');
  }

  return errors;
};
```

Servicio de Fichaje

```
// src/services/workday.service.ts
export const workdayService = {
  checkIn: async (data: CheckInForm) => {
    const response = await api.post('/jornada/check-in', data);
    return response.data;
  },

  checkOut: async (data: CheckInForm) => {
    const response = await api.patch('/jornada/check-out', data);
    return response.data;
  },

  getStatus: async () => {
    const response = await api.get('/jornada/status');
    return response.data;
  },

  getHistory: async (params?: { desde?: string; hasta?: string }) => {
    const response = await api.get('/jornada/historial', { params });
    return response.data;
  },
};
```

4. Gestión de Horarios

DTO de Creación de Horario

```
// Basado en: src/schedule/dto/create-schedule.dto.ts
export interface CreateScheduleForm {
  nombre: string;
  tipo: 'FIJO' | 'ROTATIVO' | 'PARTIDO';
  esRotativo?: boolean;
  semanasRotacion?: number;
  distribuciones: DistribucionDiaForm[];
}

export interface DistribucionDiaForm {
  diaSemana: 'LUNES' | 'MARTES' | 'MIERCOLES' | 'JUEVES' | 'VIERNES' | 'SABADO' |
'DOMINGO';
  horasDia: number;
  semanaRotacion?: number;
  franjas: FranjaHorariaForm[];
}

export interface FranjaHorariaForm {
  inicio: string; // "08:00"
  fin?: string;   // "14:00"
}
```

Servicio de Horarios

```
// src/services/schedule.service.ts
export const scheduleService = {
  create: async (data: CreateScheduleForm) => {
    const response = await api.post('/schedule', data);
    return response.data;
  },

  getAll: async () => {
    const response = await api.get('/schedule');
    return response.data;
  },

  getById: async (id: number) => {
    const response = await api.get(`/schedule/${id}`);
    return response.data;
  },

  update: async (id: number, data: Partial<CreateScheduleForm>) => {
    const response = await api.put(`/schedule/${id}`, data);
    return response.data;
  },

  delete: async (id: number) => {
    await api.delete(`/schedule/${id}`);
  },
};
```

5. Manejo de Errores y Estados

Manejo Centralizado de Errores

```
// src/utils/errorHandler.ts
export const handleApiError = (error: any): string => {
  if (error.response?.status === 400) {
    // Errores de validación del DTO
    const validationErrors = error.response?.data?.message;
    if (Array.isArray(validationErrors)) {
      return validationErrors.join(', ');
    }
    return validationErrors || 'Datos inválidos';
  }

  if (error.response?.status === 401) {
    return 'No autorizado - Inicie sesión nuevamente';
  }

  if (error.response?.status === 403) {
    return 'Acceso denegado - No tiene permisos para esta acción';
```

```
  }

  if (error.response?.status === 404) {
    return 'Recurso no encontrado';
  }

  return 'Error del servidor - Intente nuevamente más tarde';
};
```

Estados de Carga Globales

```
// src/hooks/useLoading.ts
export const useLoading = () => {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const executeWithLoading = async (operation: () => Promise<any>) => {
    try {
      setLoading(true);
      setError(null);
      const result = await operation();
      return result;
    } catch (err) {
      setError(handleApiError(err));
      throw err;
    } finally {
      setLoading(false);
    }
  };

  return { loading, error, executeWithLoading };
};
```

6. Geolocalización y Fichaje

Servicio de Geolocalización

```
// src/services/geolocation.service.ts
export const getCurrentPosition = (): Promise<GeolocationPosition> => {
  return new Promise((resolve, reject) => {
    if (!navigator.geolocation) {
      reject(new Error('Geolocalización no soportada'));
      return;
    }

    navigator.geolocation.getCurrentPosition(
      resolve,
      (error) => reject(new Error('Error obteniendo ubicación')),
```

```
      {
        enableHighAccuracy: true,
        timeout: 10000,
        maximumAge: 300000, // 5 minutos
      }
    );
  });
};

export const checkInWithGeolocation = async () => {
  try {
    const position = await getCurrentPosition();
    const checkInData = {
      latitud: position.coords.latitude,
      longitud: position.coords.longitude,
      presente: true,
    };

    return await workdayService.checkIn(checkInData);
  } catch (error) {
    throw new Error('Error de geolocalización: ' + error.message);
  }
};
```

7. Sistema de Notificaciones

WebSocket para Alarmas

```
// src/services/websocket.service.ts
export const connectWebSocket = (empresaId: number) => {
  const ws = new WebSocket(`ws://localhost:3000/alarms/${empresaId}`);

  ws.onmessage = (event) => {
    const alarm = JSON.parse(event.data);

    // Mostrar notificación al usuario
    if ('Notification' in window && Notification.permission === 'granted') {
      new Notification('Alarma de Fichaje', {
        body: alarm.message,
        icon: '/alarm-icon.png',
      });
    }

    // Actualizar estado global de alarmas
    updateAlarms(alarm);
  };

  return ws;
};
```

📊 8. Reportes y Consultas

Filtros y Paginación

```
// src/types/filters.ts
export interface WorkdayFilters {
  desde?: string; // "2024-01-01"
  hasta?: string; // "2024-01-31"
  userId?: number;
  includeDeleted?: boolean;
}

export const getWorkdayReport = async (filters: WorkdayFilters) => {
  const params = new URLSearchParams();

  if (filters.desde) params.append('desde', filters.desde);
  if (filters.hasta) params.append('hasta', filters.hasta);
  if (filters.userId) params.append('userId', filters.userId.toString());
  if (filters.includeDeleted) params.append('includeDeleted', 'true');

  const response = await api.get(`/jornada/reporte?${params}`);
  return response.data;
};
```

🎯 9. Buenas Prácticas de Integración

Validación Frontend ↔ Backend

1. **Siempre consultar el DTO** antes de crear formularios
2. **Mismas validaciones** en frontend y backend
3. **Mensajes de error** consistentes
4. **Tipos de datos** idénticos

Manejo de Estados

1. **Loading states** para todas las operaciones asíncronas
2. **Error boundaries** para manejo de errores
3. **Optimistic updates** para mejor UX
4. **Rollback** en caso de error

Seguridad

1. **Nunviar enviar empresaId** en el body
2. **Usar siempre cookies HTTP-only** para tokens
3. **Validar permisos** en frontend antes de llamar endpoints
4. **Sanitizar datos** de entrada

Performance

1. **Lazy loading** de datos grandes
2. **Caching** de datos frecuentes
3. **Paginación** en listados largos
4. **Debouncing** en búsquedas

📞 Soporte y Mantenimiento

🐛 Reporte de Errores

- **Backend**: Logs en NestJS Logger
- **Frontend**: Error boundaries + console
- **Integración**: Sistema de monitoreo

📈 Monitoreo

- **Performance**: Tiempos de respuesta
- **Uso**: Métricas de endpoints más utilizados
- **Errores**: Tasa de errores por módulo

🔄 Actualizaciones

- **Versionado**: Semántico (major.minor.patch)
- **Migraciones**: Automáticas con TypeORM
- **Backward compatibility**: Mantenida cuando sea posible

📝 Notas Finales

Este sistema está diseñado como una solución **enterprise-ready** con:

- **Arquitectura escalable** basada en microservicios
- **Seguridad multi-empresa** completa
- **Sistema de fichaje** con geolocalización
- **Horarios complejos** con rotación automática
- **Integración frontend** estandarizada

Para cualquier duda o soporte técnico, consultar la documentación específica de cada módulo o contactar al equipo de desarrollo.

Última actualización: Marzo 2026 Versión: 1.0.0